Exp 13: Write a Python Program to Implement the Minimax Algorithm for Gaming

Aim

To implement the **Minimax algorithm** in Python to determine the best possible move for a two-player game.

Objective

- To understand the working of the Minimax algorithm.
- To implement game tree search using recursion.
- To determine the optimal move for both players.
- To simulate decision-making in two-player games.
- To learn the concept of maximizing and minimizing players in AI.

Algorithm

Step 1: Start the program.

Step 2: Define an evaluation function to assign scores to terminal game states.

Step 3: Define a function to check whether the game has reached a terminal state.

Step 4: Generate all possible legal moves for the current player.

Step 5: Apply the Minimax algorithm recursively.

Step 6: If the maximum search depth is reached or the game is over, return the evaluation score.

Step 7: If the current player is the maximizing player (X), choose the move with the highest score.

Step 8: If the current player is the minimizing player (O), choose the move with the lowest score.

Step 9: Return the best score to the previous recursive call.

Step 10: Display the optimal score and stop the program.

Program

```
# Minimax Algorithm Example

# Evaluation function
def evaluate(state):
    return state
```

```
# Check whether game is over
def game_over(state):
    return abs(state) == 10

# Generate possible moves
def get_possible_moves(state):
    return [-1, 1]

# Apply move
def make_move(state, move, player):
    if player == "X":
        return state + move
    else:
        return state - move

# Minimax function
def minimax(state, depth, player):

    if depth == 0 or game_over(state):
        return evaluate(state)

    if player == "X":          # Maximizing player
        best_score = float('-inf')

        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)
            score = minimax(new_state, depth-1, "O")
            best_score = max(best_score, score)

        return best_score

    else:                      # Minimizing player
        best_score = float('inf')

        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)
            score = minimax(new_state, depth-1, "X")
            best_score = min(best_score, score)

        return best_score

# Driver code
initial_state = 0
depth = 4

result = minimax(initial_state, depth, "X")

print("Best Score:", result)
```

Sample Outpu

Best Score: 0

(The output may vary depending on the evaluation function and game state.)

Result

Thus, the Minimax algorithm was successfully implemented in Python. The algorithm recursively evaluates all possible moves and selects the optimal move by maximizing the current player's score while minimizing the opponent's score.

Exp 14 : Write a Python Program to Implement Alpha-Beta Pruning Algorithm for Gaming

Aim

To implement the Alpha-Beta Pruning algorithm in Python for efficient game tree search by eliminating unnecessary branches.

Objective

- To understand the Alpha-Beta pruning technique.
- To optimize the Minimax algorithm using pruning.
- To reduce the number of nodes evaluated.
- To determine the best possible move for a two-player game.
- To improve game search efficiency.

Algorithm

Step 1: Start the program.

Step 2: Define an evaluation function to assign scores to terminal game states.

Step 3: Define a function to check whether the game has reached a terminal state.

Step 4: Generate all possible legal moves for the current player.

Step 5: Initialize **Alpha** = -∞ and **Beta** = +∞.

Step 6: If the maximum search depth is reached or the game is over, return the evaluation score.

Step 7: If the current player is the maximizing player (X):

Find the maximum score.

Update Alpha.

If Alpha ≥ Beta, stop searching the remaining branches (Pruning).

Step 8: If the current player is the minimizing player (O):

Find the minimum score.

Update Beta.

If Beta ≤ Alpha, stop searching the remaining branches (Pruning).

Step 9: Return the best score obtained.

Step 10: Display the optimal score and terminate the program.

Program

```
# Alpha-Beta Pruning Example

# Evaluation function
def evaluate(state):
   return state

# Check whether game is over
def game_over(state):
   return abs(state) == 10

# Generate possible moves
def get_possible_moves(state):
   return [-1, 1]

# Apply move
def make_move(state, move, player):
   if player == "X":
      return state + move
   else:
      return state - move

# Alpha-Beta Pruning function
def alpha_beta_pruning(state, depth, alpha, beta, player):

   if depth == 0 or game_over(state):
      return evaluate(state)

   # Maximizing Player
   if player == "X":
      best_score = float('-inf')

      for move in get_possible_moves(state):
         new_state = make_move(state, move, player)
         score = alpha_beta_pruning(new_state, depth-1, alpha, beta, "O")

         best_score = max(best_score, score)
         alpha = max(alpha, score)

         if beta <= alpha:
            break

      return best_score

   # Minimizing Player
```

```
    else:
        best_score = float('inf')

        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)
            score = alpha_beta_pruning(new_state, depth-1, alpha, beta, "X")

            best_score = min(best_score, score)
            beta = min(beta, score)

            if beta <= alpha:
                break

        return best_score

# Driver Code
initial_state = 0
depth = 4

result = alpha_beta_pruning(initial_state, depth,
                            float('-inf'),
                            float('inf'),
                            "X")

print("Best Score:", result)
```

Sample Output

Best Score: 0

Result

Thus, the Alpha-Beta Pruning algorithm was successfully implemented in Python. The algorithm efficiently determines the optimal move by pruning unnecessary branches in the game tree, reducing the number of nodes evaluated while producing the same result as the Minimax algorithm.

Exp 15: Write a Python Program to Implement Decision Tree

Aim

To implement the **Decision Tree Classification** algorithm using Python and classify the Iris dataset.

Objective

- To understand the Decision Tree classification algorithm.
- To train a Decision Tree classifier using the Iris dataset.
- To predict the class labels of test data.
- To evaluate the classifier using accuracy.
- To analyze the prediction results.

Algorithm :

Step 1: Import the required Python libraries.

Step 2: Load the Iris dataset.

Step 3: Split the dataset into training and testing sets.

Step 4: Create a Decision Tree classifier.

Step 5: Train the classifier using the training dataset.

Step 6: Predict the class labels for the testing dataset.

Step 7: Display the predicted values.

Step 8: Calculate the accuracy of the model.

Step 9: Display the accuracy.

Step 10: Stop the program.

Program

```
# Import necessary libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load the Iris dataset
```

```
iris = load_iris()

X = iris.data
y = iris.target

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Create Decision Tree Classifier
clf = DecisionTreeClassifier(random_state=42)

# Train the model
clf.fit(X_train, y_train)

# Predict the test data
y_pred = clf.predict(X_test)

# Display predicted values
print("Predicted Output:")
print(y_pred)

# Display actual values
print("\nActual Output:")
print(y_test)

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)
```

Output:

```
array([1, 0, 2, 1, 1, 0, 1, 2, 1, 1,
       2, 0, 0, 0, 0, 1, 2, 1, 1, 2,
       0, 2, 0, 2, 2, 2, 2, 2, 0, 0,
       0, 0, 1, 0, 0, 2, 1, 0, 0, 0,
       2, 1, 1, 0, 0])

Accuracy: 1.0
```

Result

Thus, the Python program to implement the **Decision Tree Classification** algorithm was successfully executed. The model predicted the test data and achieved high classification accuracy on the Iris dataset.

Exp 16 : Write a Python Program to Implement Feed Forward Neural Network

Aim

To implement a Feed Forward Neural Network (FFNN) using Python and Keras for classifying the Iris dataset.

Objective

To understand the architecture of a Feed Forward Neural Network.

To build a neural network using Keras.

To train the network using the Iris dataset.

To classify the test data using the trained model.

To evaluate the performance of the neural network using accuracy.

Algorithm :

Step 1: Import the required libraries.

Step 2: Load the Iris dataset.

Step 3: Split the dataset into training and testing sets.

Step 4: Convert the target labels into one-hot encoded vectors.

Step 5: Create a Feed Forward Neural Network with:

One input layer

One hidden layer with ReLU activation

One output layer with Softmax activation

Step 6: Compile the model using the SGD optimizer and categorical cross-entropy loss function.

Step 7: Train the model for 50 epochs using a batch size of 10.

Step 8: Evaluate the model using the testing dataset.

Step 9: Display the testing accuracy.

Step 10: Stop the program.

Program

```
# Import necessary libraries
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import SGD
from keras.utils import to_categorical
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Convert target into one-hot encoding
y = to_categorical(y)

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Create Feed Forward Neural Network
model = Sequential()

# Hidden layer
model.add(Dense(10, input_dim=4, activation='relu'))

# Output layer
model.add(Dense(3, activation='softmax'))

# Compile the model
sgd = SGD(learning_rate=0.01)

model.compile(loss='categorical_crossentropy',
              optimizer=sgd,
              metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train,
          epochs=50,
          batch_size=10,
          verbose=1)

# Evaluate the model
scores = model.evaluate(X_test, y_test, verbose=0)
```

```
print("Accuracy: %.2f%%" % (scores[1] * 100))
```

Output

Result

Thus, the Feed Forward Neural Network (FFNN) was successfully implemented using Python and Keras. The model was trained on the Iris dataset for 50 epochs and achieved high classification accuracy on the testing data.